

Objektno programiranje v Javi

45 ur predavanj

- govornilne ure
torek 13:00-15:00 G2, 1. nad, kab št. 7
E-pošta: marjan.hericko@um.si

Sestava ocene

- 40% tedenske vaje
(vaje, kvizi, rač. preverjanje znanja)
- 40% kolokvija oz. pisni izpit
- **20% ustni zagovor**
(predhodno: opravljene vaje in
vsaj 50% pisnega/kolokvijev)

Asistenta:

- Lucija Brezočnik
- Luka Četina

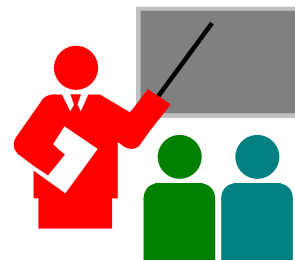
Obvezna udeležba na vajah!

<http://estudij.um.si>

1

Vsebina

- Zgodovina in značilnosti
- Sintaksa Jave
- OO in Java
- Obravnava izjem
- Grafične komponente in oblikovanje GUI
- Dogodkovni model
- Delo s vhodno / izhodnimi tokovi in serializacija
- Dostop do podatkovne baze
- Delo z omrežjem
- Sočasno izvajanje v Javi (niti)
- Zbirke objektov (Collection API)
- Javanski komponentni model (Javanska zrna)



2

Module java.base
Package java.util
Class Scanner

java.lang.Object
java.util.Scanner

All Implemented Interfaces:
Closeable, AutoCloseable, Iterator<String>

```
public final class Scanner
extends Object
implements Iterator<String>, Closeable
```

A simple text scanner which can parse primitive types and strings using regular expressions.

A Scanner breaks its input into tokens using a delimiter pattern, which by default matches whitespace. The resulting tokens may then be converted into values of different types using the various next methods.

For example, this code allows a user to read a number from System.in:

```
Scanner sc = new Scanner(System.in);
int i = sc.nextInt();
```

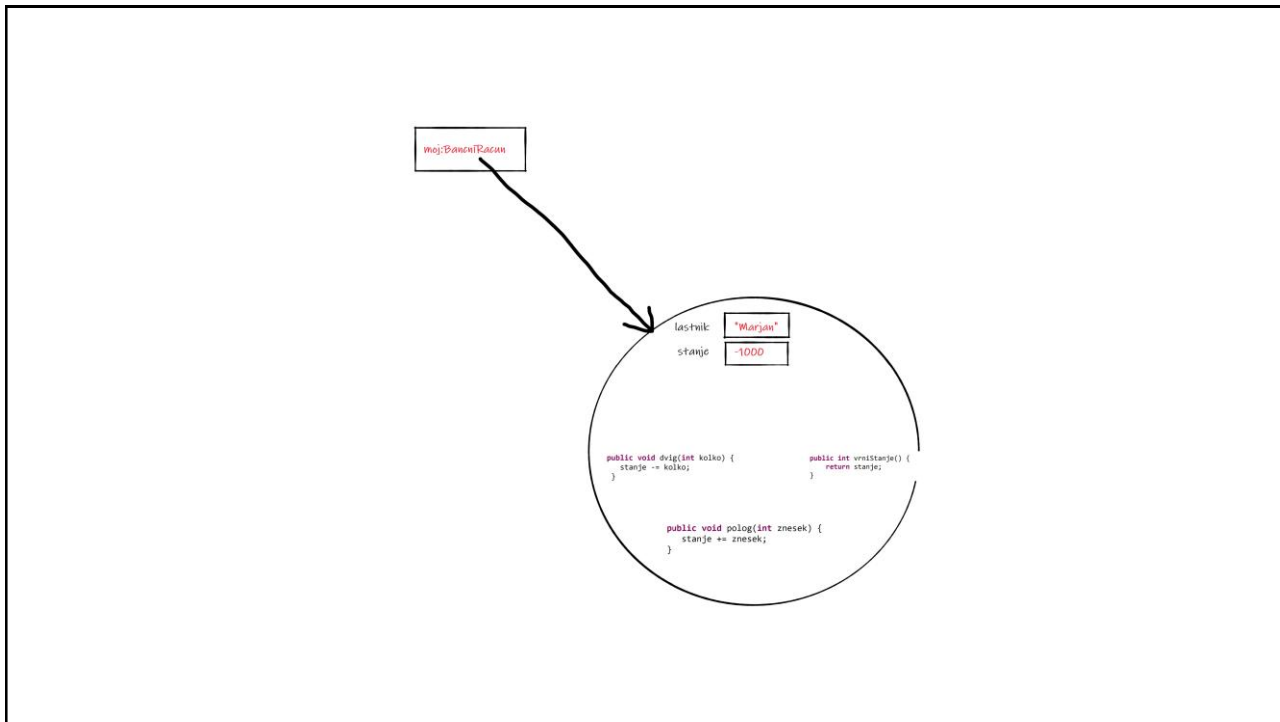
int	nextInt()	Scans the next token of the input as an int.
int	nextInt(int radix)	Scans the next token of the input as an int.
String	nextLine()	Advances this scanner past the current line and returns the input that was skipped.
long	nextLong()	Scans the next token of the input as a long.
long	nextLong(int radix)	Scans the next token of the input as a long.
short	nextShort()	Scans the next token of the input as a short.
short	nextShort(int radix)	Scans the next token of the input as a short.
int	radix()	Returns this scanner's default radix.
void	remove()	The remove operation is not supported by this implementation of Iterator.
Scanner	reset()	Resets this scanner.
Scanner	skip(String pattern)	Skips input that matches a pattern constructed from the specified string.
Scanner	skip(Pattern pattern)	Skips input that matches the specified pattern, ignoring delimiters.
Stream<String>	tokens()	Returns a stream of delimiter-separated tokens from this scanner.
String	toString()	Returns the string representation of this Scanner.

3

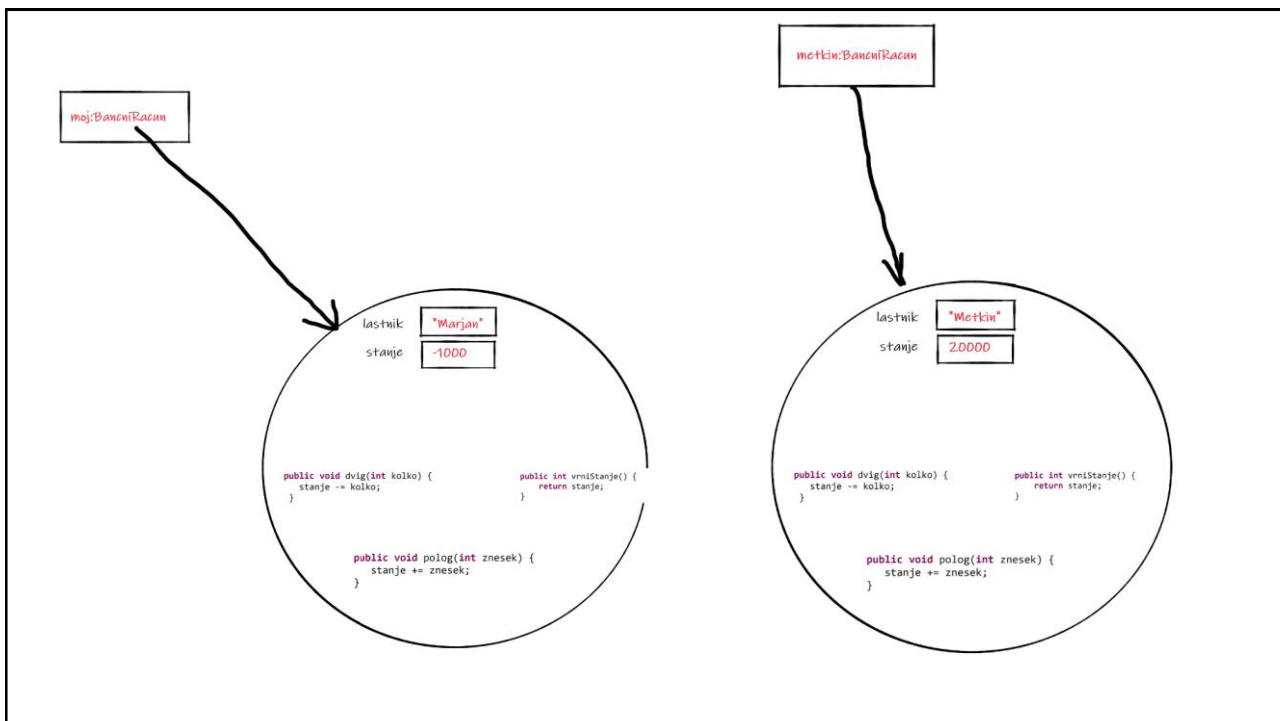
Prva predavanja

- Večji uvodni primer
 - Indeksi pri poljih od 0 dalje, dolžina atribut **length**
 - java.util.Scanner (nextInt(), nextLine())
 - izjeme (ArrayIndexOutOfBoundsException, IOException)
 - io tokovi (FileWriter(String, boolean) - ne pozabimo na flush() in close())
 - GUI – java.awt Frame (setVisible(boolean), add(Component), Button
 - dogodkovni model - vmesnik java.awt.event.ActionListener, lambda izraz
 - Kolekcije (**ArrayList**)
 - **import**
- class BancniRacun (**najprej metode!**)
- objekte (običajno) kreiramo z **new**
- konstruktorji, rezervirana besedica **this**
- preko objektnih referenc prožimo metode
- vidnost (metod in atributov) – **public, protected, private** (sicer package friendly)
- **java.lang.Object**, metoda **public String toString()**, **@Override**
- **static** označuje razredne attribute in metode

4

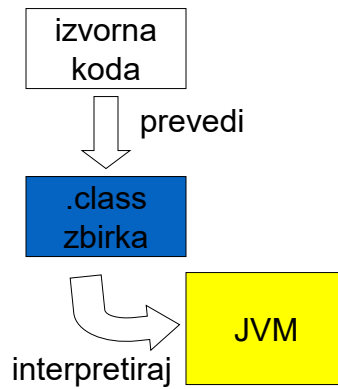


5



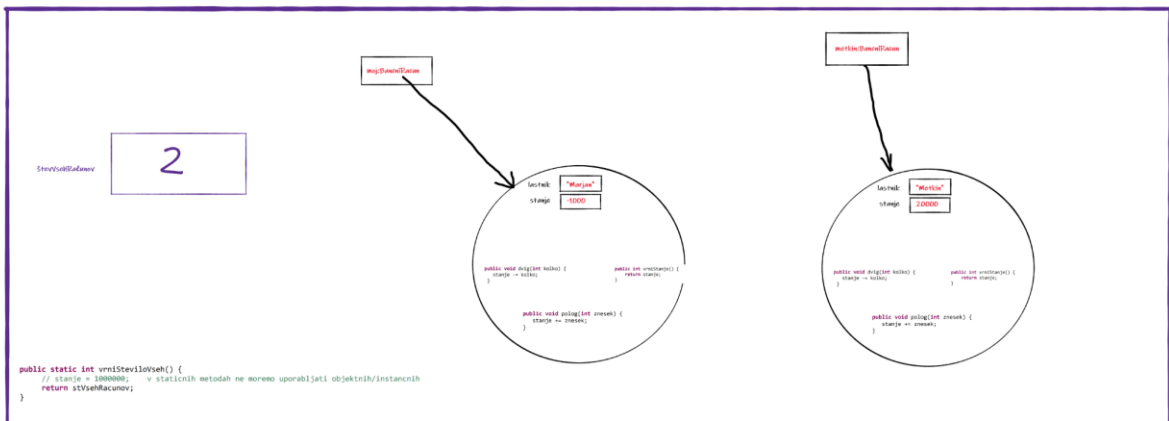
6

Vmesna koda – ključ prenosljivosti



7

BančniRacun



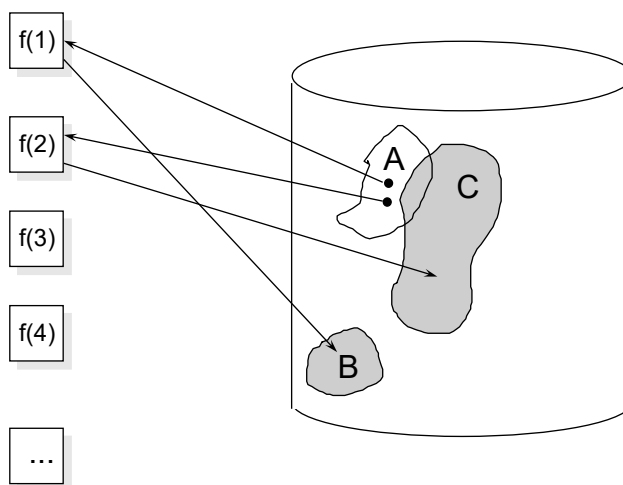
8

OO in Java

Osnove uporabe konceptov objektno orientiranega programiranja v Javi

9

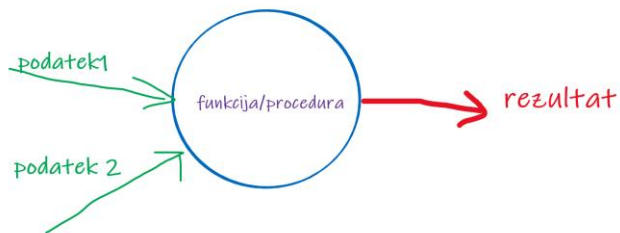
Konvencionalni sistem



Problem: spremembe v pod. strukturah

10

Proceduralno

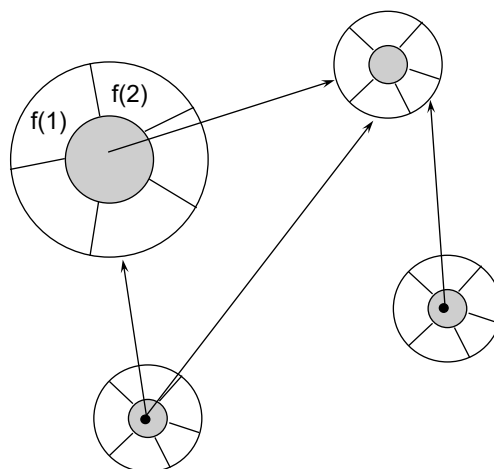


```
//funkcija/procedura(podatek1, podatek2);
```

```
starost = izracunajStarost(04-03-1980);
```

11

OO sistem

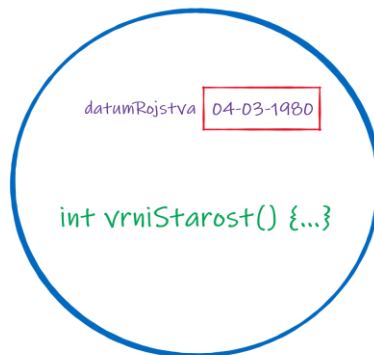


Naloga: zgraditi jasne in stabilne vmesnike

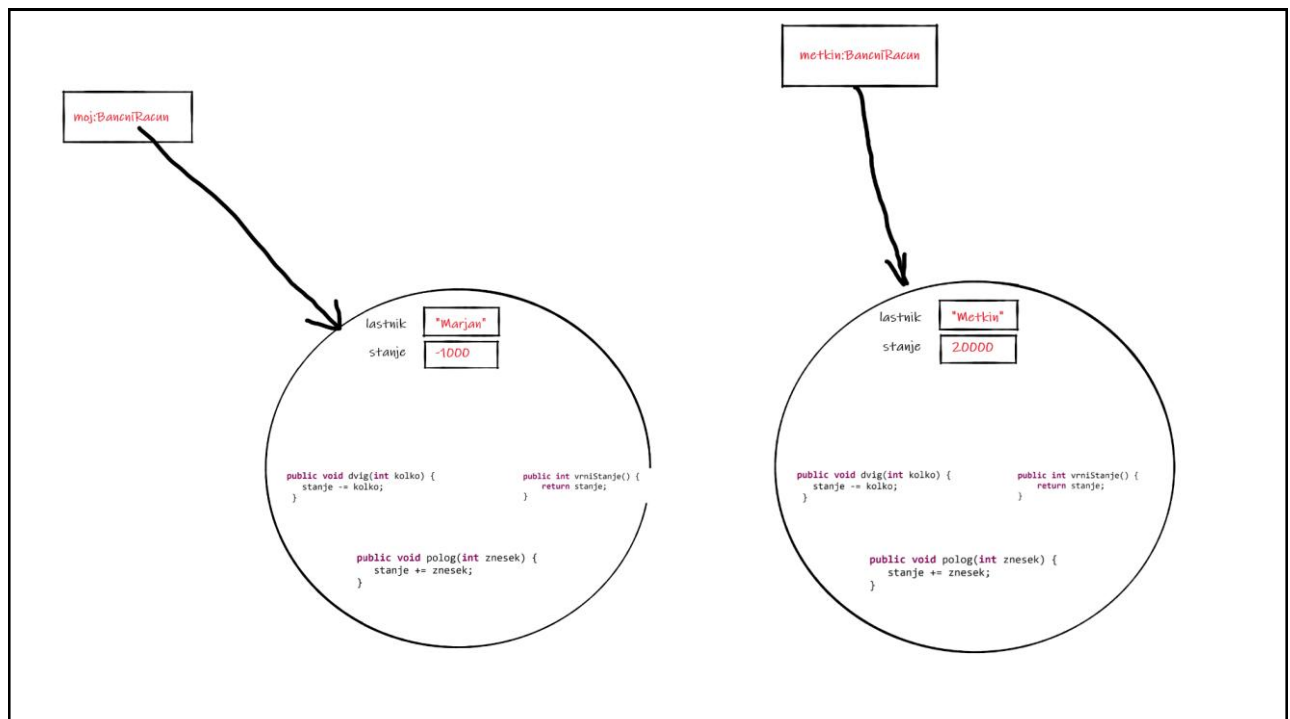
12

Objektno

janez : **Oseba**



13



14

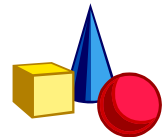
OO koncepti

- objekt ("object")
- ograževanje ("encapsulation")
- razred ("class")
- vmesnik ("interface")
- dedovanje ("inheritance")
- delegiranje ("delegation")
- komunikacija s sporočili
- polimorfizem ("polymorphism")

15

Objekti

- objekt (object) je skupek
 - podatkov oz. **atributov**
 - proceduralne logike - **metod**
- ločimo interne in vmesniške metode (funkcije)
- vmesniške metode definirajo način za komunikacijo z objektom
- interne metode mora poznati le načrtovalec objekta
- vsak objekt je enolično določen s svojo identiteto oz. identifikatorjem



16

Objektni sistemi

• Kaj je objekt?

- je **primerek** (instanca) nekega razreda
- zagotavlja **nabor operacij oz. metod**, preko katerih odjemalci delujejo nad stanjem objekta
- objekti komunicirajo s **pošiljanjem sporočil** (message-passing)
- ima **stanje** (predstavitev skrita)
- stanje določa množica atributov objekta **in** njegovih povezav z ostalimi objekti

• Za OO značilni najosnovnejši koncepti

- **ograjevanje** (encapsulation) / **abstraktni podatkovni tipi**
- **dedovanje** (inheritance)
- **identiteta objekta** (object identity)

17

Objektni način razmišljanja

• Svetilka

vkluči
izključi
svetleje
zasenči



■ Bančni Račun

dvig
polog
vrniStanje



RAZMISLIMO O ODGOVORNOSTIH!

18

Kartica CRC

(Class Responsibilities Collaboration)

Razred: Zbirka	
Nadrazredi:	
Podrazredi: GrafičnaZbirka, ZbirkaDokument, TekstovnaZbirka	
Hrani svojo vsebino	
Tiska svojo vsebino	Tiskalnik

razred, nadrazredi, podrazredi, odgovornosti, sodelovanje

19

Faze Java objekta

- deklaracija

```
ImeRazreda imeObjekta;
```

- kreiranje z new

```
imeObjekta = new ImeRazreda (seznamParametrov) ;
```

```
ImeRazreda imeObjekta =  
new ImeRazreda (seznamParametrov) ;
```

- uporaba

```
imeObjekta.imeAtributa;  
imeObjekta.imeMetode (parametri) ;
```

- uničenje, brisanje je naloga upravljalca pomnilnika, ta kliče metodo, ki jo lahko predefiniramo:

```
protected void finalize() ;
```

20

Konkretnije

BacniRacun crniFond, skbTR, ljubicaFond;

```
skbTR = new BacniRacun(500000);
crniFond = new BacniRacun(1000000);
ljubicaFond = new BacniRacun(1000000,"Metka");
```

```
skbTR.dvig(250000);
ljubicaFond.polog(245000);
crniFond.vrniStanje ();
```

```
crniFond.stanje = 2000000;
```

21

Primeri

- stvari, naprave
Letalo, Knjiga, Izdelek, Material, Del, Zgradba
- osebe/vloge
Delavec, Dobavitelj, Zastopnik, Stranka, Posrednik, Doktor, Pacient, Nadzornik, Davkoplačevalec
- dogodki
Polet, Prodaja, Prenos, Plačilo, Naročilo, Rezervacija, Dobava, Registracija, Odpoved
- pojmi, koncepti
Garancija, Faza proizvodnje, Smer razvoja, Kakovost
- kraji/lokacije
Pisarna, Skladišče, Mesto, Regija

konkretni/konceptualni

poslovni/tehnološki

22

Objekti

- konkretni / konceptualni

Centrala, Naročnik
Klic, Tarifa, Operater

- **poslovni** / **tehnološki**

Centrala, Vod, Operater, Tel. številka, Pacient,
JFrame, Thread, XMLParser, DBCursor

23

Java - dogovori glede poimenovanja

- Imena razredov pišemo z veliko začetnico
- imena objektov z malo začetnico
- standard poimenovanja:

<http://www.oracle.com/technetwork/java/codeconventions-150003.pdf>

24

UML

- Na diagramih bomo uporabljali notacijo jezika za objektno modeliranje UML (Unified Modeling Language)
- več informacij
 - www.rational.com/uml
 - OMG (Object Management Group)
www.omg.org/techprocess/meetings/schedule/Technology_Adoptions.html

25

Notacija UML

razred vs. objekt vs. vloge

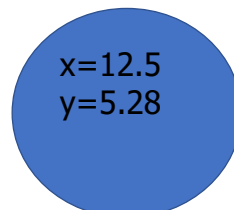
razred

Točka
-x:Real -y:Real
+postavi(x:Real, y:Real) +premakni (dx:Real, dy:Real)

objekt

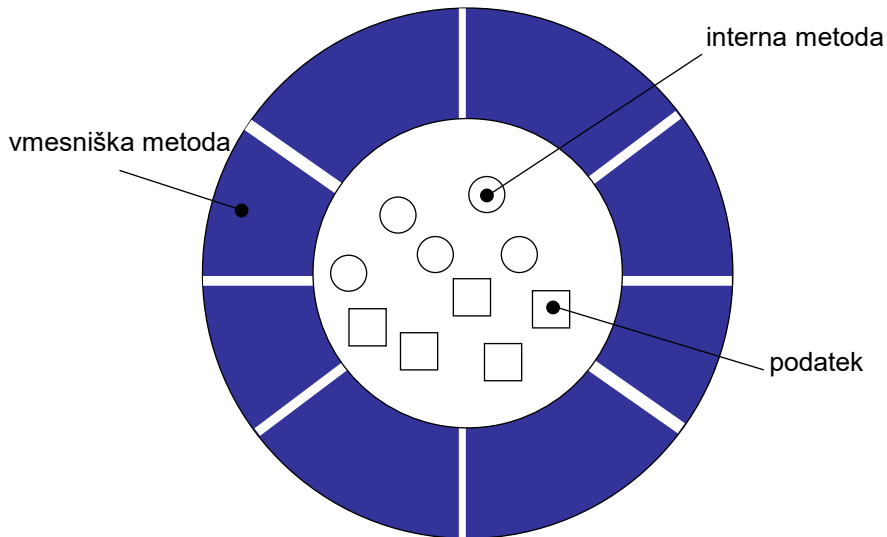
<u>t1:Točka</u>
x=12.5 y=5.28

t1:Točka



26

Gradniki objektov

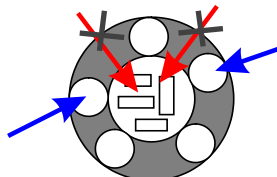


27

“encapsulation”

Ograjevanje

Abstrakcija vzpodbuja in zahteva jasno ločitev **(zunanjega) vmesnika** in **interne implementacije** objekta.

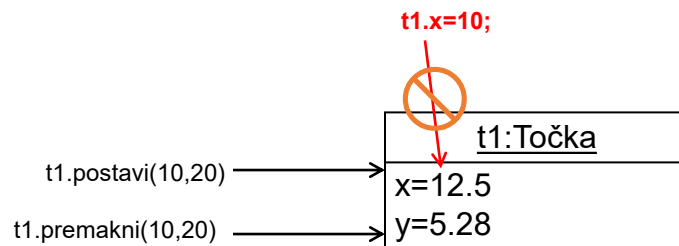


- **skrije podrobnosti** implementacije (skrivanje informacij) in zagotavlja modularnost
- objekt ima **viden (javni) vmesnik** in **skrito (privatno) implementacijo**

28

Ograjevanje

- skrivanje implementacije
- dostop do podatkov preko proženja vmesniških metod
- omejitev verižnih reakcij



29

Ograjevanje

- pomaga zmanjšati medsebojno funkcionalno odvisnost (omeji verižne reakcije)
- spodbuja skrivanje informacij
- skrije uporabnikom objekta implementacijske podrobnosti
- pogojuje, da objekte obravnavamo kot celoto oz. osnovni gradnik
- podatki, atributi so del ustreznega objekta
- zagotavlja, da vsa manipulacija nad objekti poteka preko njegovih metod
- objekt ne more direktno spreminjati stanja drugega objekta

30

Spodbujanje lepega pristopa

- **zbirni jezik**
(simbolični kod - linearna preslikava strojnega koda, Rxyz, JMP)
- **višji programski jeziki** (še vedno GOTO)
- **strukturirano programiranje**
(brez goto)
- **hibridni objektni jeziki**
(objekti, funkcije)
- **čisti objektni jeziki** (objekti)
- **komponente** (jasni vmesniki)

31

"class"

Razred

- več podobnih objektov specificiramo z istim splošnim opisom
- opis objektov imenujemo **razred** (class)
- vsak objekt je **primerek**, predstavnik (**instance**) nekega razreda
- razred definira implementacijo metod ter attribute
- razred kot nekakšen vzorec, po katerem se kreirajo objekti

32

Abstract Data Type (ADT)

- *Razred objektov* – vsak objekt pripada nekemu ADT - razredu objektov
- Skrivanje informacij
 - objekti so na voljo le preko operacij zunanjega vmesnika, definiranega z ADT
 - podrobnosti implementacije, podatkovne strukture in elementi shranjevanja navzven niso vidni
- Celovitost: definirane so vse potrebne operacije
- *Kreiranje objektov* ("Object Creation"/ *Instantiation*): s proženjem operatorja `new` ustreznega razreda

33

Deklaracija razredov

```
[komentarji za dokumentacijo]
[modifikatorji] class ImeRazreda
[extends NadRazred]
[implements SeznamVmesnikov]
{
    TeloRazreda
}
```

34

Telo razreda

- deklaracije spremenljivk
- privzete vrednosti razrednih spremenljivk (static initializers)
- deklaracije konstruktorjev
- deklaracije metod

zaporedje je poljubno, ni nujno, da so grupirani, vendar je zaradi čitljivosti bolje, če upoštevamo zgornje zaporedje

35

Definiranje metod

```
[komentarji za dokumentacijo]
[modifikatorji]
    TipRezultata imeMetode (SeznamArgumentov)
{
    telo metode
}

abstract TipRezultata imeMetode (SeznamArgumentov) ;
```

36

Deklaracija razreda

```
class Krog {
    // instance variables
    int x,y; // koordinate
    int r;    // polmer
    // instance methods
    double obseg () {return 2*3.14*r;}
    double ploscina() {return 3.14*r*r;}
}
```

37

Deklaracije

- zelo podobno C++ razredom
- med razredi le enkratno dedovanje
- **vse metode in atributi (spremenljivke) morajo pripadati nekemu razredu**
- ni prostih globalnih metod (funkcij),
ne globalnih spremenljivk
- razredi imajo konstruktorje (lahko le privzetega)
- v Javi ni destruktorjev

38

Določitev vidnosti (ograjevanje)

```
class Krog {
    // instance variables
    private int x,y; // koordinate
    private int r;    // polmer
    // instance methods
    public double obseg ()
        {return 2*3.14*r;}
    public double ploscina()
        {return 3.14*r*r;}
}
```

private, protected, public oz. če nič: "package-friendly"

39

Dostop do spremenljivk in metod

	public	protected	private	privzeto friendly - package
vsi	✓			
podrazredi	✓	✓		
razred sam	✓	✓	✓	✓
razredi istega paketa	✓	✓		✓

40

Razred

```

class Krog {
    private int x,y;
    private int r;

    Krog(int polmer) {
        r=polmer; x=0; y=0;
    }

    Krog(int polmer, int kjeX,int kjeY ){
        r=polmer; x=kjeX; y=kjeY;
    }

    public double obseg () {
        return 2*3.14*r;}

    public double ploscina()
    {return 3.14*r*r;}

    public void premakni (int dx, int dy) {
        x+=dx; y+=dy;
    }
}

```

Krog
-x,y:int
-r:int
+obseg()
+ploscina()
+premakni (dx:int, dy:int)

41

Metode – konstruktorji

```

class Krog {
    private int x,y;    // koordinate
    private int r;      // polmer
    Krog(int polmer) {
        r=polmer; x=0; y=0;
    }
    Krog(int polmer, int kjeX,int kjeY ){
        r=polmer; x=kjeX; y=kjeY;
    }
}

```

več konstruktorjev, isto ime, kaj pa vračajo?

42

Kreiranje objektov

Konstruktorji ("constructors")

- imajo isto ime kot razred
- lahko jih je več - imajo različne parametre (signature)
- ne vračajo ničesar
operator new **vrne referenco na kreiran objekt**
- v konstruktorju se inicializira objekt tega razreda
- konstruktor brez argumentov je privzeti konstruktor, ki ga prožijo podrazredi, zato je klic **super()** odveč, saj za to avtomatično poskrbi prevajalnik, torej je klic **super (parametri)** aktualen le, če ga kličemo z argumenti,
- če ne specificiramo konstruktorja, prevajalnik avtomatično kreira konstruktor brez argumentov, ki preprosto kliče **super()**

43

Kreiranje primerkov

Krog mali;

mali = new Krog(2);

mali:Krog
x=0 y=0 r=2

Krog veliki = new Krog (10,5,5);

veliki:Krog
x=0 y=0 r=2

new Krog(2,4,5);

:Krog
x=4 y=5 r=2

44

- instance variables - member variables
 - lahko so poljubnega tipa
 - lahko imajo privzete vrednosti
 - če privzete vrednosti ne navedemo, bodo inicializirane na:
 - 0 pri numeričnih tipih
 - false pri logičnih (boolean)
 - null pri vseh ostalih tipih

45

Metode (instančne oz. objektne)

- **instance method**
v C++ terminologiji member function
- določen mora biti **tip vrednosti, ki jo vrača**
(razen pri konstruktorjih), **sicer void**
- spremenljivke, deklarirane znotraj metod, so lokalne spremenljivke
- lokalna spremenljivka ne more prekriti druge lokalne spremenljivke ali parametra (argumenta)

46

RAZREDNI ATRIBUTI in metode

47

Razredne spremenljivke /atributi

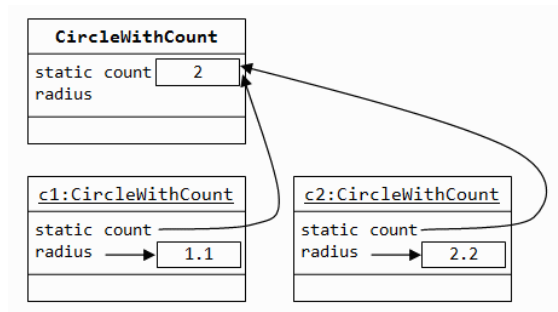
static

- skupni podatki za vse primerke (ista vrednost)
- uporabljajo/dostopajo lahko vsi primerki tega razreda
- inicializacija se izvrši samo enkrat (**static { }**)

Razredne metode

- znotraj razreda delujejo lahko le nad razrednimi spremenljivkami tega razreda
- glede metod istega razreda - prožijo lahko le razredne
- uporabimo jih lahko tudi brez primerkov razreda (preko imena razreda, kar je dobra praksa)
- lahko pa pošiljajo sporočila drugim objektom

48



49

Dobra/slaba praksa

- do razrednih metod in spremenljivk/atributov dostopamo preko imena razreda npr.

```
BancniRacun.povecajLimit();
BancniRacun.stRacunov = 100;
```

sicer lahko tudi preko objektov, a je zavajajoče

```
BancniRacun crniFond;
```

```
...
```

```
crniFond.povecajLimit();
crniFond.stRacunov=100;
```

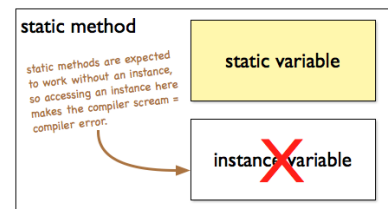
50

Razredne metode

razredne (statične) metode **ne morejo uporabljati ne-statičnih** (objektnih / instančnih) **spremenljivk in metod** (te pripadajo posameznim objektom)

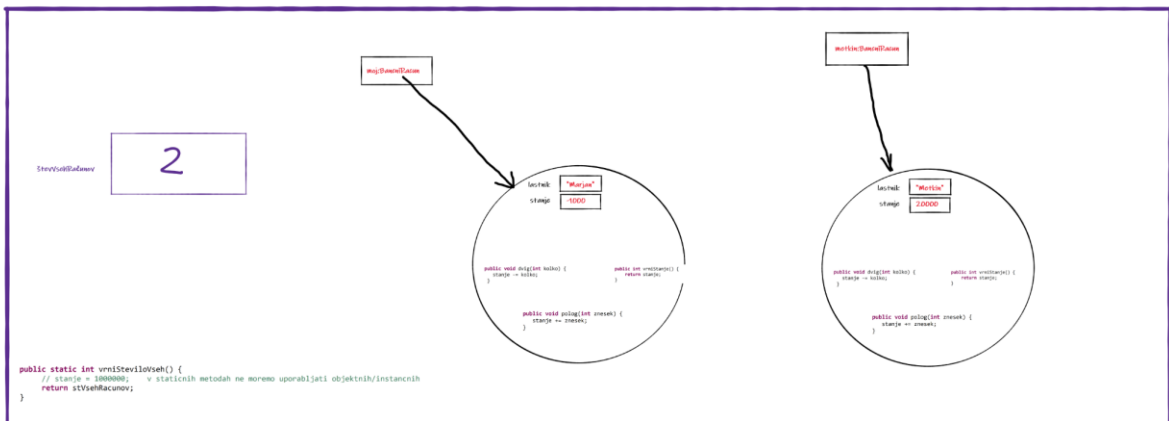
Če to skušamo narediti, prevajalnik javi napako

non-static variable [or method] *Name* cannot be referenced from a static context



51

BančniRacun



52

static

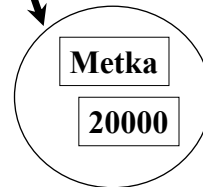
stVsehRacunov

100

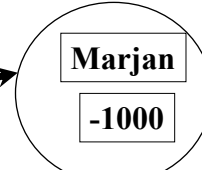
```
BancniRacun.stVsehRacunov = 100;
moj.stVsehRacunov = 100;
metkin.stVsehRacunov = 100;
```

objektne/instančne

metkin



moj



53

Rezervirani besedi *static* in *final***static**

- pomeni iste vrednosti za vse primerke
- pri metodah pomeni razredno metodo
- dostopamo lahko kar preko razreda in ne preko primerkov razreda (objektov)

Primer:

```
System.out.println
(" Gremo na odmor");
```

final

- omogoča deklaracijo konstant
- takšnih metod ne moremo "povoziti", so končne
- z njimi izboljšamo zmogljivost programa (prevajalnik)

Primer:

```
final float PI=3.14;
```

54

Privzete vrednosti razrednih spremenljivk

"static initializer"

```
class DelovniDnevi {  
    static String[] imena;  
    static int zapSt[];  
    static {  
        imena = new String[5];  
        imena[0]="Ponedeljek";  
        imena[1]="Torek"; imena[2]="Sreda";  
        imena[3]="Četrtek"; imena[4]="Petek";  
        for (int i=0; i<10;)  
            zapSt[i]=++i;  
    }  
}
```

55

Običajna uporaba **this**

- kakšen argument metode ima isto ime kot atribut razreda
- lokalna spremenljiva ima isto ime kot eden izmed atributov
- v enem konstruktorju je potrebno prožiti drug konstruktor

56

```

class Krog {
    double r; // polmer

    // argument z istim imenom!
    void setPolmer (double r)
        {this.r = r;}

    // lok. sprem z istim imenom!
    void printPolmer100krat () {
        for (int r=0; r<100; r++) {
            System.out.println(this.r);
        }
    }
}

```

57

```

class Krog {
    double x,y; // koordinate
    double r; // polmer
    // instance methods
    double obseg () {return 2*3.14*r;}
    double ploscina() {return 3.14*r*r;}

    // uporaba univerzalnega konstruktorja
    Krog (double r) { this (0,0,r);}
    Krog (Krog c) { this (c.x, c.y, c.r);}
    Krog (double xx, double yy, double rr) { x=xx;
        y=yy; r=rr;}
}

```

58

Ustvarjanje in brisanje objektov

Constructor

- ima isto ime kot razred
- lahko jih je več in imajo različne argumente
- ne vrača ničesar
- v prvi vrstici lahko kličemo `super()`
- v njem inicializiramo objekt za ta razred

Finalizer

- **`finalize()`**
- izvede se tik preden zbiralec odpadkov odstrani objekt
- ni predvidljiv čas izvedbe
- z njim počistimo za objektom

59

Struktura programov

60

Struktura programov

enota prevajanja (izvorna zbirka) lahko vključuje:

- stavek **package**
- stavke **import**
- deklaracije razredov
- deklaracije vmesnikov

61

Enota prevajanja

- v vsaki izvorni zbirki je lahko **le en public** razred (ki ima **isto ime kot zbirka**) oz. public razred mora biti v zbirki z istim imenom
- ostali razredi so dejansko podporni
- razred z istim imenom (ni nujno)
- ali nujno, da je sploh kateri javen?

62

Organiziranje razredov v pakete

63

Združevanje razredov

Package

- povemo kam spada zbirka
- imenike ločimo s piko

Primer:

```
package si.hmi;
```

Import

- vključi zunanjo kodo
- pot lahko navedemo točno ali z *

Primer:

```
import java.awt.Panel;  
import java.util.*;
```

64

Imena

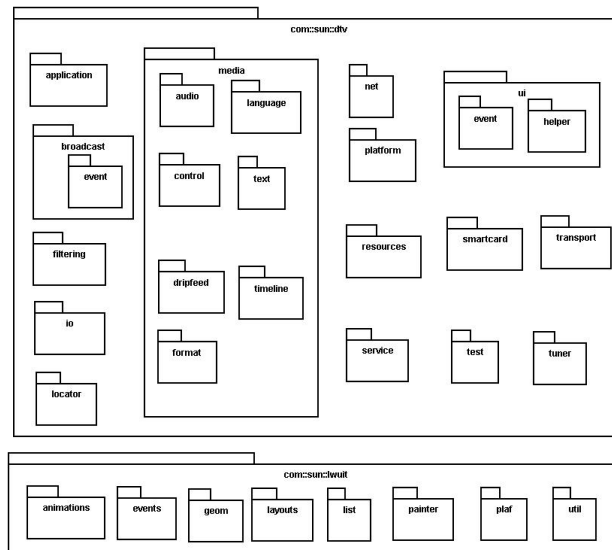
- kako poimenovati?
predlog: obratno ime domene, npr.
si.mhi
si.telekom
- BancniRacun je sedaj v:
package si.mhi. banka;

65

Uporaba razredov iz knjižnic

- vključim ustrezen razred ali celotno knjižnico
import java.util.Vector;
import java.util.*;
- lahko se sklicujem le z imenom razreda ali pa s celotno hierarhijo
new Vector();
new java.util.Vector();

66

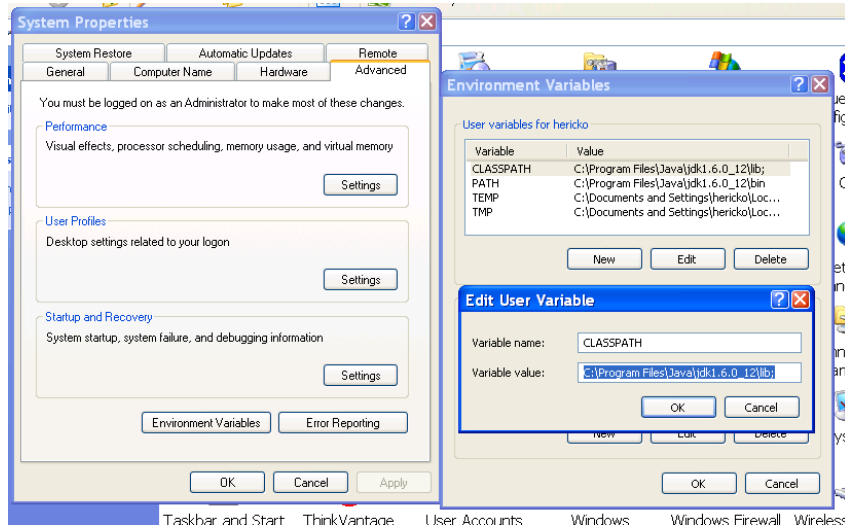


67

classpath

- pomembna spremenljivka okolja
- korenski imenik za iskanje ustreznih knjižnic
- lahko podamo tudi pri zagonu
`java -classpath .;c:\opj2020 si.hmi.BR`
 ali prevajanju
`javac -cp .;c:\opj2020 Banka.java`

68



69

Objektna tehnologija

- ni vezana le na eno področje
- vpliva na vsa področja informatike
(jezike, pod. baze, operacijske sisteme, metode razvoja, prenova poslovnih procesov), standardizacija!!!
- integrira vse koristne in uporabne tehnike svojih predhodnikov
- osnova za razvoj na osnovi komponent
(Component Based Development)

70

Objektna paradigma

- Simula 67 - ADT
- Smalltalk - 72
- Po 1980 so se objekti začeli seliti iz raziskovalnih laboratorijev v realni svet
- Rodil se je C++, Smalltalk je postal splošno uporaben
- Začetki uporabe v fazi implementacije, šele kasneje se je OO uveljavil tudi v ostalih fazah procesa razvoja IS in programske opreme

71

Objektna paradigma

- objektnega programskega jezika se ni težko naučiti
- objektni jeziki ne vsiljujejo (nekateri vzpodbujajo) objektnega razmišljanja, čeprav podpirajo OO koncepte
- težko se je naučiti uporabljati prednosti, ki jih objektni (komponentni) jezik nudi

72

Objektna paradigma

Objektna orientacija ne
ZAGOTAVLJA prednosti -
prednosti le **NUDI!**

73

Osnove Jave

74

Nastanek Jave

- projekt Green (Sun, 1990)
(razvoj jezika za porazdeljene sisteme elektronskih naprav
zahtevana zanesljivost, nizki stroški, preprostost, neodvisnost od procesorja, povezljivost)
- začeli z razširitvami in na osnovi C++
(ni šlo)
- 1991 - Oak (hrast)
izdelali so operacijski sistem, orodje, vmesnik, več čipov
kasneje projekt: video na zahtevo (Time Warner)
- 1995 - HotJava
(pregledovalnik v celoti napisan v Javi)



75

Indikator popularnosti (TIOBE index)

Feb 2026	Feb 2025	Change	Programming Language	Rating	Change
1	1		Python	21.81%	-2.08%
2	4	▲	C	11.05%	+1.22%
3	2	▼	C++	8.95%	-2.82%
4	3	▼	Java	8.12%	-2.54%
5	5		C#	6.83%	+2.71%
6	6		JavaScript	2.92%	-0.85%
7	10	▲	Visual Basic	2.85%	+0.81%
8	15	▲	R	2.19%	+1.14%
9	7	▼	SQL	1.93%	-0.93%
10	9	▼	Delphi/Object Pascal	1.88%	-0.29%
11	30	▲	Perl	1.67%	+1.19%
12	11	▼	Fortran	1.64%	-0.12%
13	14	▲	PHP	1.34%	+0.20%
14	13	▼	Rust	1.32%	-0.14%
15	12	▼	Scratch	1.30%	-0.25%
16	8	▼	Go	1.23%	-1.03%
17	24	▲	Ada	1.14%	+0.43%
18	16	▼	MATLAB	1.13%	+0.14%
19	17	▼	Assembly language	1.10%	+0.15%
20	23	▲	Kotlin	1.05%	+0.29%

<https://www.tiobe.com/tiobe-index/>

76

Povpraševanje, plače

<https://www.itjobswatch.co.uk/jobs/uk/java.do>

Java ⓘ
UK

The table below provides summary statistics and salary benchmarking for jobs requiring Java skills. It covers permanent job vacancies from the 6 months leading up to 25 February 2026, with comparisons to the same periods in the previous two years.

UK	6 months to 25 Feb 2026	Same period 2025	Same period 2024
Rank	70	43	26
Rank change year-on-year	▼ -27	▼ -17	▼ -9
Permanent jobs citing Java	2,061	2,590	5,699
As % of all permanent jobs in the UK	2.97%	5.46%	7.57%
As % of the Programming Languages category	15.93%	15.83%	20.45%
Number of salaries quoted	1,316	1,603	3,859
10 th Percentile	£42,500	£41,900	£40,000
25 th Percentile	£55,000	£53,750	£52,500
Median annual salary (50 th Percentile)	£75,000	£75,000	£70,000
Median % change year-on-year	-	+7.14%	-6.67%
75 th Percentile	£100,000	£112,500	£95,000
90 th Percentile	£125,000	£137,500	£118,750
UK excluding London median annual salary	£65,000	£60,000	£60,000
% change year-on-year	+8.33%	-	-



Java Top 30 Co-Occurring Skills & Capabilities

For the 6 months to 25 February 2026, job vacancies citing Java also mentioned the following skills and capabilities in order of popularity. The figures indicate the absolute number of co-occurrences and as a proportion of all permanent job ads with a requirement for Java.

1	920 (44.64%)	AWS	16	363 (17.61%)	Mentoring
2	789 (38.28%)	Agile	16	363 (17.61%)	Test Automation
3	722 (35.03%)	Python	17	352 (17.08%)	Problem-Solving
4	704 (34.16%)	Software Engineering	18	350 (16.98%)	React
5	686 (33.28%)	CI/CD	19	342 (16.59%)	Security Cleared
6	519 (25.18%)	DevOps	20	311 (15.09%)	Degree
7	472 (22.90%)	Kubernetes	20	311 (15.09%)	TypeScript
8	455 (22.08%)	Microservices	21	261 (12.66%)	GCP
9	454 (22.03%)	JavaScript	22	250 (12.13%)	Terraform
10	438 (21.25%)	SQL	23	240 (11.64%)	Containerisation
11	425 (20.62%)	Spring	24	239 (11.60%)	Linux
12	420 (20.38%)	Azure	25	237 (11.50%)	AI
13	398 (19.31%)	Full-Stack Development	26	236 (11.45%)	Computer Science
14	382 (18.53%)	Docker	27	235 (11.40%)	C#
15	377 (18.29%)	Spring Boot	28	228 (11.06%)	REST

77

Java danes

- Java platforma
 - [Java SE](#), [Java EE](#), [Java ME](#)
 - varnost
 - RMI, IDL
 - Media API
 - komponente (Enterprise) Java Beans
 - Java procesorji (interpreter vgrajen na čipu)
- Java na strežnikih
- ogrinja poslovnih objektov v Javi
- mobilne aplikacije

78

JDK 25 JDK 21

Java SE Development Kit 25.0.2 downloads

JDK 25 binaries are free to use in production and free to redistribute, at no cost, under the [Oracle No-Fee Terms and Conditions \(NFTC\)](#).

JDK 25 will receive updates under the NFTC, until September 2028, a year after the release of the next LTS. Subsequent JDK 25 updates will be licensed under the [Java SE OTN License \(OTN\)](#) and production use beyond the [limited free grants](#) of the OTN license will [require a fee](#).

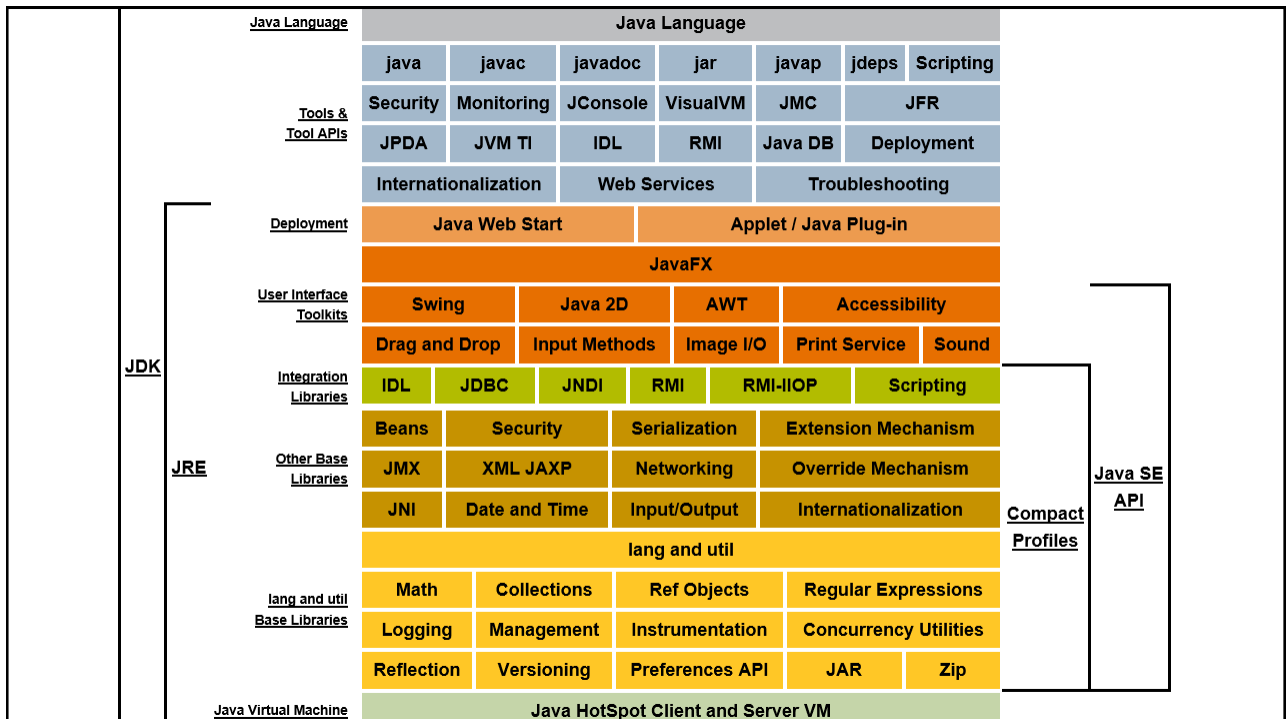
Linux macOS Windows

Product/file description	File size	Download
ARM64 Compressed Archive	203.36 MB	https://download.oracle.com/java/25/latest/jdk-25_linux-aarch64_bin.tar.gz (sha256)
ARM64 RPM Package	202.96 MB	https://download.oracle.com/java/25/latest/jdk-25_linux-aarch64_bin.rpm (sha256) (OL 9 GPG Key)
x64 Compressed Archive	205.51 MB	https://download.oracle.com/java/25/latest/jdk-25_linux-x64_bin.tar.gz (sha256)
x64 Debian Package	175.97 MB	https://download.oracle.com/java/25/latest/jdk-25_linux-x64_bin.deb (sha256)
x64 RPM Package	205.07 MB	https://download.oracle.com/java/25/latest/jdk-25_linux-x64_bin.rpm (sha256) (OL 9 GPG Key)

Documentation Download

<https://www.oracle.com/java/technologies/javase-downloads.html>

79



80

- JRE (Java Run-Time Environment)
 - JVM
 - knjižnice (API) java in javax
- JDK (Java Development Kit)
 - JRE
 - orodja (java, javap, javadoc, ...)

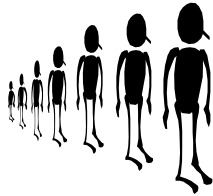
81

Značilnosti jezika (1/4)



82

Značilnosti jezika (2/4)



PORAZDELJEN

INTERPRETIRAN

ROBUSTEN



83

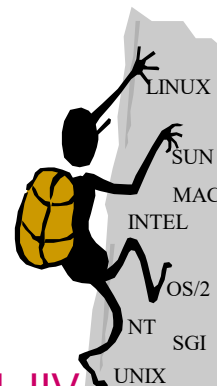
Značilnosti jezika (3/4)



VAREN

ARHITEKTURNO NEODVIŠEN

PRENOSLJIV



84

Značilnosti jezika (4/4)



VISOKO ZMOGLJIV



DINAMIČEN



VEČOPRAVILEN

85

Preprostost

- cilj: enostavno učenje (malo konstruktov)
- blizu in domač večini programerjev (zato podoben C-ju)
- izogibanje težavam in problemom (zato izločili kazalce, goto, header zbirke, struct, union, večkratno dedovanje)
- je čisti OO jezik (ne hibridni kot C++)

86

Objektno - orientiran

- vse razen osnovnih podatkovnih tipov - primitiv (boolean, int, short, char, byte, float, double) so objekti
- usmerjanje k objektnemu razmišljanju in načrtovanju na osnovi odgovornosti oz. operacij, ki jih posamezni objekt zagotavlja
- ne omogoča "hibridov" - prostih funkcij ipd.
- obširne knjižnice razredov v paketih (packages)

87

Osnovni podatkovni tipi

- byte (8 bitov)
- short (16 bitov)
- int (32 bitov)
- long (64 bitov)
- float (32 bitov)
- double (64 bitov)
- char (16 bitov)
- boolean



R A

vse ostalo mora biti objekt

nima struct
nima typedef
nima #define
nima kazalcev

88

Interpretiran

- Java prevajalnik generira vmesno kodo (byte code), ki je neodvisna od procesorja
- potrebujemo interpreter in run-time sistem, ki skupaj tvorita Java navidezni stroj (Java Virtual Machine)
- potencialna slabost in problem – počasnost
 - zato: JIT - Just In Time Compiler
 - z uporabo native funkcij izgubimo neodvisnost
- dobra stran: za razvijalca prijaznejše okolje - hitrejše delo

89

Porazdeljen

- zaradi naravnosti k razvoju omrežnih aplikacij - podpora delu z omrežnimi viri
- posebna knjižnica (package java.net)
Sockets, URL, HttpHandler
- nič več klasičnih sistemov odjemalec/strežnik
izmenjava vlog
- RMI, IDL, WS, REST...

90

Robusten

- zmanjšuje možnosti oblikovanja nezanesljivih programov - še vedno pa velja
"A fool with a tool is still a fool."
- odstranitev konstruktov (kazalcev...)
- način dela s pomnilnikom
dodeljevanje pomnilnika je dinamično, problem fragmentacije in izgubljenih referenc, garbage collection skrbi za dinamični del pomnilnika
statični del - definicije razredov in tabele metod
- je močno tipiziran jezik, preverjanje v času prevajanja
- obravnava izjem

91

Varen

- še posebej pomembno, če se že imenuje "jezik omrežja"
- prepletenost s prej naštetimi lastnostmi
- ni kazalcev - ne moremo v pomnilnik
- preverjanje razredov
- preprečevanje nepooblaščenega dostopa do lokalnih virov
- nič ni 100%
- Java ponuja razumen kompromis glede ignoriranja nevarnosti, ki jih s seboj prinaša omrežje

92

Arhitekturno neodvisen, prenosljiv

- neodvisna vmesna koda - zlogokod ("byte code")
- Java programe lahko izvajamo na vseh sistemih, za katere obstaja (imamo) JVM Java Virtual Machine
 - javanski navidezni stroj
- koristi tako razvijalcem kot uporabnikom

93

Visoko zmogljiv

- Java se interpretira, zato nikoli ne bo tako hitra kot C
- *kar vsekakor zadostuje za interaktivne grafične in omrežne aplikacije, ker so pogosto v stanju čakanja na uporabnika ali podatke*
- JIT (Just in Time) prevajanje
- pri snovanju vmesne kode (zlogokoda) so imeli v mislih učinkovito generiranje strojnega koda
- statični del pomnilnika (def. razredov in tab. metod)

94

Večopravilen

- pomembno pri grafičnih in omrežnih aplikacijah
- npr. hkrati predvajanje zvoka, prikaz slik, prenašanje zbirk ...
- vgrajena podpora večopravilnosti
razred Thread v paketu java.lang
(metode za kreiranje, upravljanje, zaključevanje, preverjanje stanja)
- konstrukt synchronized preprečuje napake

95

Dinamičen

- razredi se naložijo takrat, ko jih potrebujemo (tudi preko omrežja)
- preverjanje pripadnosti objektov posameznim razredom v času izvajanja omogoča dinamično povezovanje razredov v delujoč sistem
- komponente - “on the fly” integracija

96

Java platforma

Java platformo sestavlja pet enot:

- jezik
- osnovne knjižnice razredov
- struktura vmesne kode (zbirk “.class”)
- definicija vmesne kode (byte code definition)
- specifikacija navideznega stroja

zadnje tri točke podpirajo idejo:
“Napiši enkrat, poženi kjerkoli”
(“Write once, run anywhere.”)

97

- JRE (Java Run-Time Environment)
 - JVM
 - knjižnice (API) java in javax
- JDK (Java Development Kit)
 - JRE
 - orodja (java, javap, javadoc, ...)

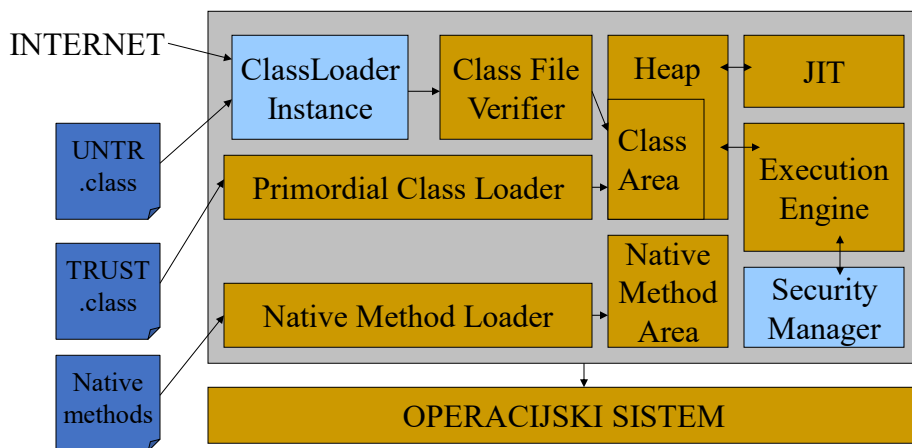
98

Kako delujejo javanski programi?

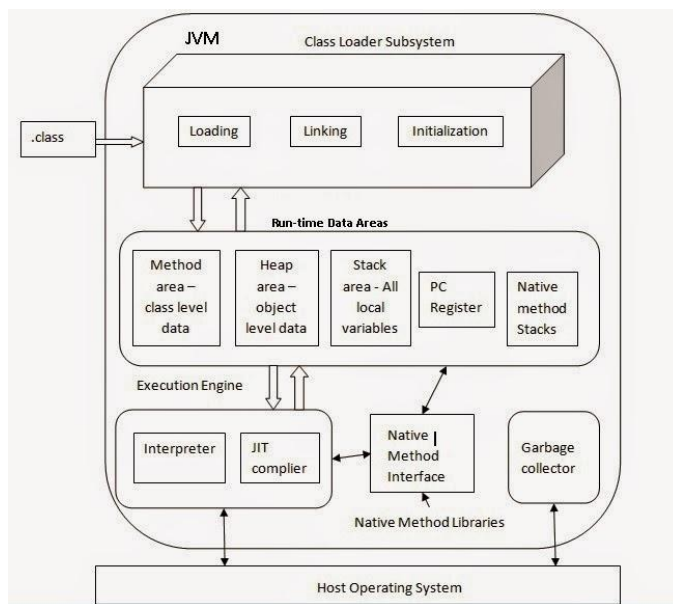
Prenosljiva zlogovna koda, navidezni stroj, upravljanje pomnilnika

99

Java navidezni stroj

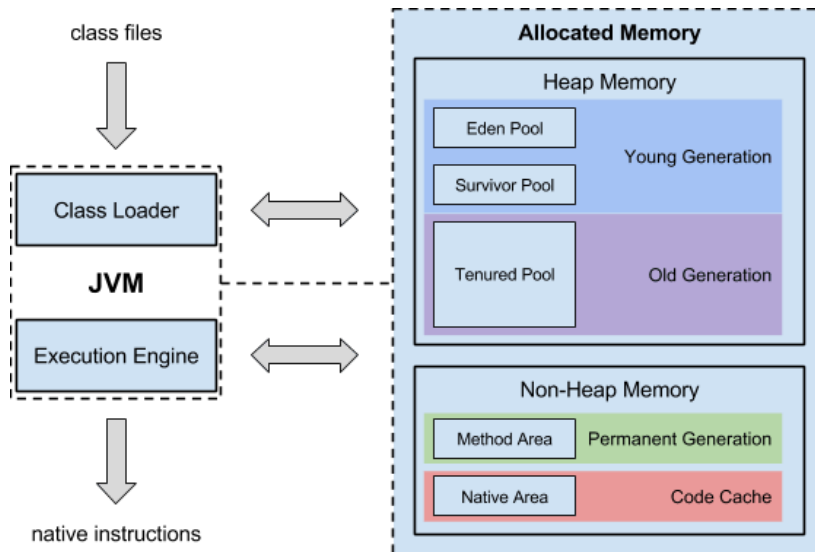


100



<http://k-mingle.blogspot.si/2015/04/jvm-architecture.html>

101



<https://anturis.com/blog/java-virtual-machine-the-essential-guide/>

102

Vmesna koda (Bytecode)

Sestavljena je iz štirih delov:

- mnemonic ukaza
- koda ukaza
- število argumentov velikosti enega byta, ki neposredno sledijo kodi tega ukaza
- opis

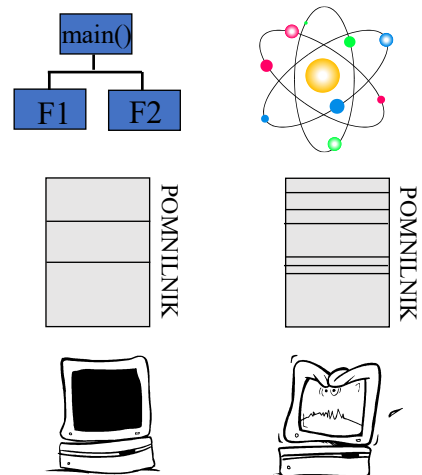
Primer:

Ukaz	Koda	Arg.	Opis
sipush	17	1	16 bitno predznačeno vrednost da na sklad kot integer

103

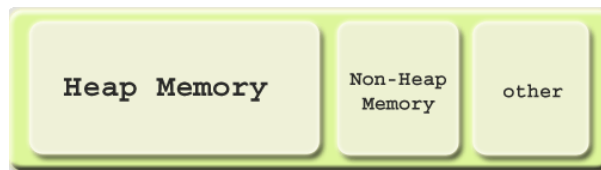
Upravljanje s pomnilnikom

- dodeljevanje pomnilnika je dinamično
- problem fragmentacije in izgubljenih referenc
- JVM razdeli pomnilnik na statičen in dinamičen
- za dinamičen del skrbi zbiralec odpadkov



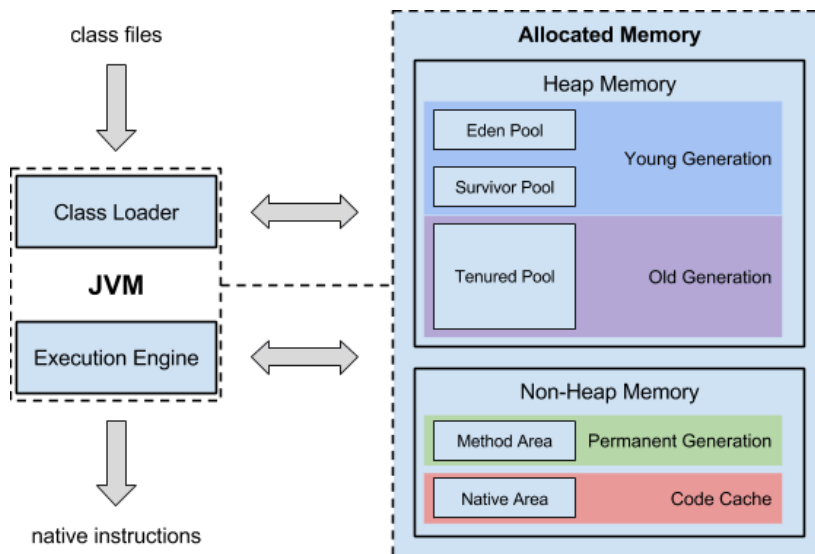
104

JVM - pomnilnik



- **garbage collector**
skrb za pomnilnik (sproščanje)

105



<https://anturis.com/blog/java-virtual-machine-the-essential-guide/>

106

Zbiralec odpadkov

- je del JVM
- skrbi za objekte v pomnilniku
- je nit z najnižjo prioriteto
- izboljša zmogljivost



107

Java

- 1995 - HotJava
- 1995 JDK verzija 1.0
- 1997 JDK verzija 1.1

specifikacije vs. implementacije

- 1998 Java 2 platforma
- 1999 Java 2 EE

Ločimo:

- Java Enterprise Edition (Java EE) **Jakarta**
- Java Standard Edition (Java SE)
- Java Micro Edition (Java ME), JavaFX, Java Embedded

108

Npr.

- Java Platform, SE (Standard Edition)
<http://java.sun.com/javase/>

rast števila razredov in vmesnikov

```

212
504
1781
2130
2738   (1.4)
....
4012

```

- Java Platform, EE (Enterprise Edition)
<http://www.oracle.com/technetwork/java/javasee/overview/index.html>
<http://java.sun.com/javasee/>
- Java Platform, ME (Micro Edition)
<http://java.sun.com/javame/>

109

Java Development Kits	Codename	Release	Packages	Classes
Java SE 8 with JDK 1.8.0	---	2014	217	4,240
Java SE 7 with JDK 1.7.0	Dolphin	2011	209	4,024
Java SE 6 with JDK 1.6.0	Mustang	2006	203	3,793
Java 2 SE 5.0 with JDK 1.5.0	Tiger	2004	166	3,279
Java 2 SE with SDK 1.4.0	Merlin	2002	135	2,991
Java 2 SE with SDK 1.3	Kestrel	2000	76	1,842
Java 2 with SDK 1.2	Playground	1998	59	1,520
Development Kit 1.1	---	1997	23	504
Development Kit 1.0	Oak	1996	8	212

110

Java SE JDK

Java Standard Edition
Java Development Kit

pregled osnovnih orodij

111

Nekatera orodja

• Osnovna orodja

- javac
- java
- javadoc
- appletviewer
- jar
- jdb
- javah
- javap
- **jre**

Java IDL orodja

- tnameserv
- idltojava
(posebej)

RMI orodja

- rmic
- rmiregistry
- rmid
- serialver

Orodja za internacionalizacijo

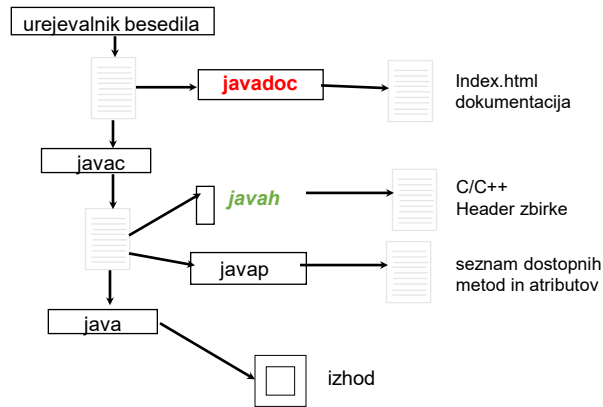
- native2ascii

Orodja za varnost

- keytool
- jarsigner
- policytool

112

Pomembnejša orodja



113

anotacije v komentarjih za generiranje dokumentacije

```

1  /**
2   * osnovni razred s podporo
3   */
4  public class BancniRacun
5
6
7      private String lastn
8      private int stanje;
9  /**
10
  
```

Class BancniRacun

java.lang.Object
↳ BancniRacun

public class BancniRacun
razred java.lang.Object

osnovni razred s podporo za drži; poleg denarja in denar do stanja

Method Detail

optimizirana

public int optimizirana(int i, float d)

Parameters:

- lastnik - ime komitenta
- zacStanje - zaceto stanje na BR

114

nekaterne anotacije za dokumentacijo

@author
@deprecated
@exception
@param
@return
@throws
@version

/** komentarji za doc
*/

115

Sintaksa Java

podatkovni tipi, rezervirane besede,
operatorji, polja, kontrolni stavki

116

Razjasnitev pojmov

spremenljivka
literal-konstante
operator
rezervirane besede

komentar
stavek
blok

117

Osnovni podatkovni tipi

-  byte (8 bitov)
 -  short (16 bitov)
 -  int (32 bitov)
 -  long (64 bitov)
 - float (32 bitov)
 - double (64 bitov)
 - char (16 bitov) 
 - boolean (1 bit)
- $R + A = RA$

```
nima struct
nima typedef
nima #define
nima kazalcev
```

118

Reserved Words in Java

Primitive Types	Modifiers	Declarations	Control Flow	Miscellaneous
1. Boolean 2. byte 3. char 4. short 5. int 6. long 7. float 8. double 9. void	1. public 2. protected 3. private 4. abstract 5. static 6. final 7. transient 8. volatile 9. synchronized 10. native	1. class 2. interface 3. enum 4. extends 5. implements 6. package 7. throws	1. if 2. else 3. try 4. catch 5. finally 6. do 7. while 8. for 9. continue 10. break 11. switch 12. case 13. default 14. throw 15. return	1. this 2. new 3. super 4. import 5. instanceof 6. null 7. true 8. false 9. strictfp 10. goto 11. assert 12. const

119

Operatorji

++ samodejno povečanje	> je večje
-- samodejno zmanjšanje	< je manjše
+ unarni plus, seštevanje	>= večje ali enako
- unarni minus, odštevanje	<= manjše ali enako
* množenje	== je enako
/ deljenje	!= ni enako
% ostanek deljenja	?: pogojno testiranje

120

Operatorji

- ! ne (not)
- & in (boolean and)
- ^ XOR
- | ali (boolean or)
- && pogojni in (ovrednoti prvi izraz...)
- || pogojni ali
- ~ inverz na biti (bitwise inversion operator)

121

Polja

- polje je objekt


```
float[] cena; //polje cen
```
- velikost dodelimo dinamično


```
cena=new float[100]; //hranimo sto cen
```

```
public float VrniCeno() {
    return cena[st_art];
}
```
- Indeksi tečejo od 0


```
String [][] tabela; // polje za tabelo
```

```
tabela=new
```
- lahko so večdimenzionalna


```
String[st_vrstic][st_stolpcev];
```

122

Kontrolni stavki

if
switch
for
while
do while

break
continue

123

stavek if

```
if (pogoj) {  
    stavek 1  
    stavek 2  
} else stavek
```

```
int dosezeno_tock;  
boolean prijava_ok;  
boolean opravi1;  
if ((dosezeno_tock>=50) &&  
    prijava_ok)  
    opravi1=true;  
else opravi1=false;
```

124

stavek switch

```

switch (barva) {
    case 0:
        System.out.println("črna");
        break;
    case 1:
        System.out.println("rjava");
        break;
    case 2:
        System.out.println("modra");
        break;
    default:
        System.out.println("neznana");
}

switch (izraz) {
    case vrednost1: stavek1;
    case vrednost2: stavek2;
    default: privzet_stavek;
}

```

125

stavek for

```

for (začetna_vrednost; pogoj; sprememba_vrednosti)
{
    stavek1
    stavek2
}

for (i=0; i<=20; i++)
    System.out.println("Vrednost s premenljivke i: " + i);

```

126

stavek while

```
while (pogoj) {  
    stavek1  
    stavek2  
}  
  
int i=0;  
while (i<=20) {  
    System.out.println("Vrednost spremenljivke i: " + i);  
    i++;  
}
```

127

stavek do while

```
do {  
    stavek1  
    stavek2  
} while (pogoj);  
  
int i=0;  
do {  
    System.out.println("Vrednost i: " + i);  
    i++;  
} while (i<=20)
```

128

Stavki za prenehanje izvajanja

break

```
while (znak != '\n') {
    znak=beri_znak;
    if (znak==iskani)
        break;
}
```

continue

```
while (znak != '\n') {
    znak=beri_znak;
    if (znak==iskani) {
        st_znakov++;
        continue;
    }
}
```

129

Prekinitev

- continue – gre na preverjanje pogoja (pri for pred tem izvede “tretji del”)
- break – prekinitev izvajanja zanke – izhod iz zanke
- break kam; // prekinitev več zank hkrati

```
zunanja: for ( ....)
           for (...)
           for (...)
               break zunanja;
```

130